

Resumo

Este relatório apresenta a análise comparativa de três arquiteturas de redes neurais convolucionais (CNN) aplicadas ao conjunto de dados MNIST de dígitos manuscritos. O objetivo foi avaliar acurácia de teste, tempo de treinamento e inferência, uso de memória e complexidade em termos de parâmetros e camadas. Os experimentos foram conduzidos em TensorFlow/Keras, com três épocas de treinamento e validação interna de 10% dos dados de treino. Os principais achados indicam que, embora todas as arquiteturas alcancem acurácias elevadas (>98%), há diferenças significativas em termos de eficiência de tempo e memória, o que deve guiar escolhas em contextos com restrições de recursos .

Introdução

As redes neurais convolucionais são amplamente empregadas em tarefas de reconhecimento de imagens devido à sua capacidade de extrair características locais e hierárquicas. O dataset MNIST, contendo 60.000 imagens de treino e 10.000 de teste de dígitos manuscritos (28×28 pixels, escala de cinza), serve como benchmark clássico para avaliar arquiteturas iniciais de CNN. Este trabalho implementa três variações arquiteturais, visando comparar desempenho e custo computacional em um cenário controlado. A motivação inclui determinar trade-offs entre complexidade e precisão em ambientes com recursos limitados .

Metodologia

Pré-processamento

- **Carregamento de dados:** Utilizou-se `mnist.load_data()` do Keras.
- **Normalização:** Os pixels foram escalonados para `[0,1]` dividindo por 255.0.
- **Redimensionamento:** As imagens foram reformuladas para formato (28, 28, 1) para entrada em camadas Conv2D.
- **Codificação das labels:** Empregou-se one-hot encoding para as 10 classes (`to_categorical`) .

Função de mensuração de desempenho

- Definiu-se função auxiliar que mensura tempo de treinamento e teste, bem como pico de memória durante essas etapas, através de `time.time()` e `tracemalloc` .

- Durante treinamento, registra-se o tempo total para completar as épocas e o pico de memória alocado. Durante avaliação, mede-se tempo de inferência no conjunto de teste e pico de memória correspondente .

Configurações de treinamento

- **Épocas:** 3
- **Batch size:** 128
- **Validação interna:** 10% dos dados de treino
- **Otimização e compilação:** Otimizador Adam, perda de entropia cruzada categórica, métrica de acurácia .

Descrição das arquiteturas testadas

1. CNN 1

- Camada Conv2D com 32 filtros 3×3, função de ativação ReLU, input_shape=(28,28,1).
- MaxPooling2D 2×2.
- Flatten.
- Dense de 64 unidades com ReLU.
- Dense de saída com 10 unidades e softmax.
- Complexidade: 5 camadas principais, ~347.146 parâmetros.
- Implementação e compilação conforme trecho de código .

2. CNN 2

- Camada Conv2D com 64 filtros 3×3, ReLU, input_shape=(28,28,1).
- MaxPooling2D 2×2.
- Dropout de 25% após pool.
- Flatten.
- Dense de 128 unidades com ReLU.
- Dropout de 50% antes da camada de saída.
- Dense de saída com 10 unidades e softmax.
- Complexidade: 7 camadas principais, ~1.386.506 parâmetros.
- Adição de dropout busca reduzir overfitting potencial; código conforme .

3. CNN 3

- Duas camadas Conv2D sequenciais de 32 filtros 3×3, ambas com ReLU, input_shape=(28,28,1).
- MaxPooling2D 2×2.
- Flatten.
- Dense de 256 unidades com ReLU.

- Dense de saída com 10 unidades e softmax.
- Complexidade: 6 camadas principais, ~1.192.042 parâmetros.
- Estrutura visa extrair características mais profundas antes do pooling, conforme implementação .

Resultados

Os experimentos foram executados conforme descrito, e os principais resultados estão compilados a seguir:

- **CNN 1:**
 - Acurácia de teste: 0,9828 (98,28%).
 - Tempo de treinamento total: ~19,08 s.
 - Pico de memória durante treinamento: ~2,45 MB.
 - Tempo de inferência no teste: ~0,76 s.
 - Pico de memória durante teste: ~0,09 MB.
 - Parâmetros: 347.146; camadas: 5 .
- **CNN 2:**
 - Acurácia de teste: 0,9823 (98,23%).
 - Tempo de treinamento total: ~44,85 s.
 - Pico de memória durante treinamento: ~2,75 MB.
 - Tempo de inferência no teste: ~1,13 s.
 - Pico de memória durante teste: ~0,08 MB.
 - Parâmetros: 1.386.506; camadas: 7 .
- **CNN 3:**
 - Acurácia de teste: 0,9877 (98,77%).
 - Tempo de treinamento total: ~53,90 s.
 - Pico de memória durante treinamento: ~3,03 MB.
 - Tempo de inferência no teste: ~1,57 s.
 - Pico de memória durante teste: ~0,09 MB.
 - Parâmetros: 1.192.042; camadas: 6 .

Os resultados completos foram coletados em estrutura tabular via pandas DataFrame, conforme trecho final de código .

Discussão

- **Trade-off acurácia vs. recursos:**

- A CNN 3 apresentou a maior acurácia (98,77%), porém exige maior tempo de treinamento (~54 s) e maior uso de memória (~3,03 MB).
- A CNN 1, embora mais simples, alcançou acurácia próxima (98,28%) com tempo e memória significativamente menores (~19 s, 2,45 MB).
- A CNN 2 tem maior número de parâmetros e tempo intermediário (~45 s), mas não superou a CNN 3 em acurácia, alcançando 98,23%.
- Em cenários com restrição de tempo ou hardware modesto, CNN 1 pode ser preferível, pois oferece boa precisão com baixo custo computacional. Em aplicações que demandam máxima acurácia e dispõem de recursos adequados, CNN 3 pode ser escolhida.
- **Impacto do dropout:**
 - A inclusão de dropout em CNN 2 não resultou em melhoria de acurácia em relação à CNN 1, mas aumentou complexidade e tempo, sugerindo que, para MNIST (tarefa relativamente simples), modelos mais profundos sem dropout podem ser mais efetivos.
- **Número de épocas:**
 - Apenas três épocas foram usadas, atendendo a validação de convergência rápida em MNIST. É possível que mais épocas não melhorem substancialmente a acurácia dada a rápida saturação observada nas curvas de validação (val_accuracy >98% já na primeira ou segunda época) .
- **Generalização para outros datasets:**
 - MNIST é relativamente simples; em conjuntos com maior complexidade (ex.: CIFAR-10, imagens em cores), arquiteturas mais profundas e técnicas de regularização podem ter impacto distinto.

Conclusão

Este relatório demonstrou que arquiteturas CNN relativamente simples podem atingir alta acurácia em MNIST com diferentes compromissos entre precisão e eficiência computacional. A CNN 1 destaca-se por oferecer precisão competitiva com baixo custo de tempo e memória, sendo recomendada para ambientes restritos. A CNN 3 obtém a melhor acurácia, embora com recursos superiores. A CNN 2, apesar de dropout, não superou as demais em acurácia/eficiência, sugerindo que, para tarefas de baixa complexidade, aumentar profundidade ou parâmetros sem propósito claro pode não ser vantajoso. Em projetos reais, é essencial balancear requisitos de precisão com restrições de infraestrutura, além de considerar explorações adicionais de arquitetura e parâmetros para datasets mais complexos.

Referências

- Dataset MNIST: Y. LeCun, C. Cortes, “MNIST handwritten digit database.”
- TensorFlow/Keras API para construção de CNN: documentação oficial TensorFlow.